



Making AI Happen in Testing Projects

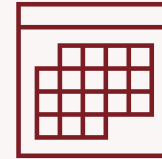
Calling GenAI APIs, running a local LLM, and building RAG from scratch — for test engineers

Abbas Ahmad · UAESTQB

One-day · hands-on · ~1/4 theory, then lab

Today's plan

- 1 · Intro — call GenAI APIs (OpenAI & Anthropic) from Python
- 2 · Local LLM — run Llama with Ollama; it answers from memory... and gets it wrong
- 3 · RAG from scratch — read a PDF, embed it, retrieve by cosine similarity
- 4 · RAG + LLM — feed the right chunk back; same question, now correct
- Wrong → right: that contrast is the whole point of today



Intro → Local LLM → RAG → RAG + LLM

Who this is for & why local matters

Run modern AI on your own machine — and stay in control.

- For test engineers, architects and QA leads — no AI background needed
- Privacy: your specs and test data never leave your machine
- Cost control: no per-token cloud bills, no surprise limits
- Transparency: you see how retrieval and answers are produced
- Local-first — add cloud APIs only when they help

What is an LLM?

- An LLM predicts the next word, again and again, from what came before
- It learns patterns of language — not a database of true facts
- Strong at drafting, summarizing, rewriting and classifying text
- It can sound confident and still be wrong (hallucination)
- So in QA we ground it in real documents — and always check it



Tokens → LLM → next-token probabilities

Tokenization: how models read text

Models don't see words — they see tokens.

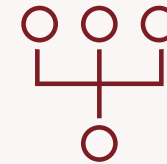
- Text is split into tokens — sub-words, each mapped to a number
- 'testing' $\approx$ 1 token; 'parametrization' splits into several
- Rule of thumb: 1 token $\approx$ 4 characters $\approx$ $\frac{3}{4}$ of a word (English)
- Context window = how many tokens the model can read at once
- Tokens drive cost and limits — you pay per token, long inputs get cut off



Text split into sub-word tokens

Talking to LLMs through APIs

- We call two providers from Python: OpenAI (GPT) and Anthropic (Claude)
- Prompt = your question; system prompt = the model's standing instructions
- Temperature: low = focused and repeatable, high = more varied
- Reasoning models “think” longer before answering — better for hard problems
- The model is stateless — for multi-turn chat you resend the whole history



System prompt + prompt → model → reply

Read the documentation

The fastest way to keep up: go to the source.

- **The single most useful skill: learn to read the API docs**
- The docs are the source of truth — providers and models change often
- OpenAI docs: platform.openai.com/docs
- Anthropic / Claude docs: docs.anthropic.com
- Find parameters, limits and new features yourself — don't guess



Docs = source of truth

Local LLM: it answers from memory

- We run a small Llama model on the laptop with Ollama — no cloud
- It loads locally: private, offline, no per-token bills
- We ask it a real ISTQB exam question
- It answers from memory — and gets it wrong
- A small model doesn't “know” your material — it guesses



Local model answers from memory → wrong

Embeddings & vector space

Turning meaning into geometry.

- An embedding turns text into a vector — a list of numbers
- Similar meaning → vectors that sit close together
- This lets us search by meaning, not exact keywords
- ‘login fails’ can match ‘authentication error’ — no shared words
- Embeddings are the engine behind retrieval in RAG



Similar concepts cluster in vector space

The problem RAG solves

- A raw LLM only knows its training data — not your documents
- Ask about your spec and it guesses → hallucination (we just saw this)
- Retraining the model every time your docs change is impractical
- RAG = hand the model the right context at question time
- Result: answers grounded in your real documentation

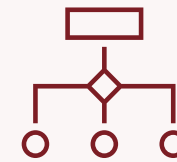


LLM guessing vs. retrieved context

The RAG pipeline, simplified

Index once, then retrieve + generate for every question.

- Ingest — read the PDF and pull out its text
- Chunk — split the text into small, overlapping passages
- Embed — turn each chunk into a vector, saved as a simple list
- Retrieve — embed the question, pick the closest chunk by cosine similarity
- Generate — the LLM answers from that chunk and cites the source



Ingest → Chunk → Embed → Retrieve → Generate

RAG + LLM: now it gets it right

- Same small Llama model, same ISTQB question as before
- This time we add the retrieved chunk to the prompt
- Now it answers correctly — grounded in the document, not memory
- Same model, better input: the context made the difference
- That wrong → right flip is the whole point of RAG



Same question + right context → correct answer

Grounding & the QA mindset

The QA mindset applied to AI outputs.

- Every AI answer should cite the source chunk it used
- Traceability: question → retrieved context → answer
- Check both: did we fetch the right context, and is the answer correct?
- Keep a human in the loop — never ship AI output unreviewed
- You are still the tester — the AI is a fast assistant, not the authority



Question → Context → Answer → Reviewer

AI won't replace test engineers

- But test engineers who can design, control and assess AI tools...
- ...will be the ones shaping the future of quality engineering.
- Stay curious. Stay critical. Keep the human in the loop.

Thank you — let's talk. · Questions?